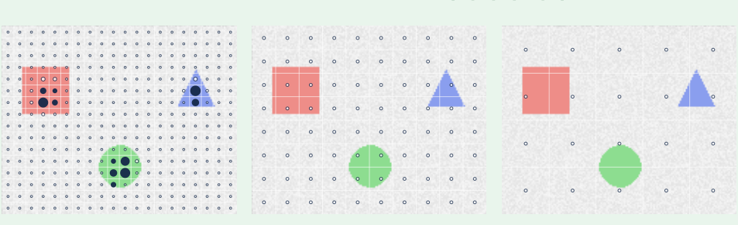
새싹 학습 노트 Day15 · 2026.11.06 · Vision·멀티모달 AI 응용 oneday 요약

객체 탐지와 YOLO 학습·추론·추적

오늘 한 문장 YOLO는 이미지를 한 번 보고 격자 칸마다 박스 후보를 낸다. conf와 NMS로 후보를 걸러 "무엇이(Class) 어디에(Box) 얼마나 확실하게(Confidence)" 있는지 남기고, 정답과의 IoU로 채점하고, 영상에서는 같은 객체를 Track ID로 잇는다.

1 한 장으로 보는 YOLO 격자 칸마다 후보를 내고, 두 번 걸러 남긴 뒤, 정답과 비교해 채점한다

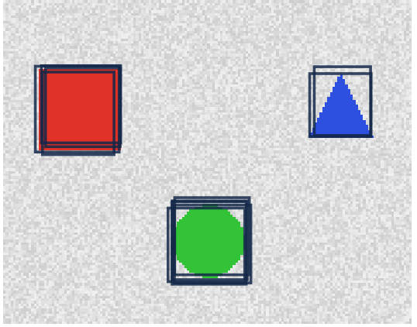
Backbone·Neck·Head 한 번 보고 격자마다 예측



420 개 후보

칸마다 박스 4개 + 클래스 점수 3개. 점이 클수록 점수가 높다(● 0.25 초과)

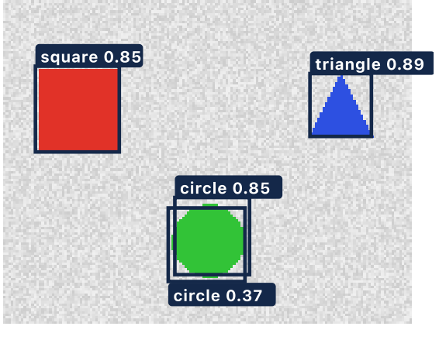
→ **conf로 거르기** `conf=0.25`



11 개

점수 낮은 후보를 버림. 같은 객체에 박스가 여러 개 겹쳐 있다

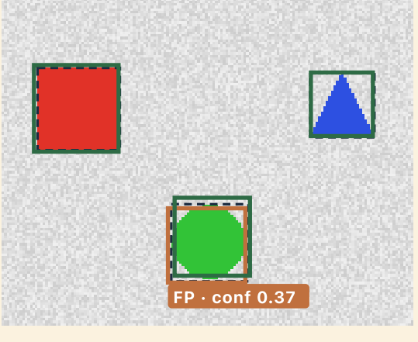
→ **NMS로 중복 지우기** `iou=0.7`



4 개 = Results

겹친 박스 중 점수가 가장 높은 것만 남김

→ **정답과 비교해 채점** `평가 IoU ≥ 0.5`



TP 3 · FP 1

점선 = 정답 3개, **초록 = TP**, **주황 = FP**. NMS 기본값 0.7에서 남은 중복 1개가 FP가 됐다

응용 val 이미지 1장(정답 3개)을 공식 추론 경로 그대로 실행한 결과입니다. 입력 160×128 → 격자 20×16 + 10×8 + 5×4 = 420칸.

꼭 알고 갈 YOLO 개념

1. **격자 칸마다 후보를 하나씩 낸다.** 이미지를 한 번 보고(One-stage) 칸마다 박스 숫자 4개와 클래스별 점수를 냅니다. 객체가 3개여도 후보는 420개입니다.
2. **격자는 세 가지 크기다.** 촘촘한 격자는 작은 객체, 성긴 격자는 큰 객체를 맡습니다. Neck이 여러 크기의 특징을 섞는 이유입니다.
3. **그래서 두 번 거른다.** conf는 점수 낮은 후보를, NMS는 같은 객체에 겹친 중복을 지웁니다. 정답과 비교하는 평가 IoU는 채점할 때만 씁니다.

하루 내내 지키는 약속

1. **train**으로 배우고, conf·checkpoint는 **val**로만 고르고, **test**는 마지막 확인용으로 남긴다
2. 비교할 때는 **한 번에 하나만** 바꾸고, 입력·가중치·imgsz·장치·버전을 함께 기록한다
3. **세 가지 기준을 섞지 않는다.** conf는 후보 점수, NMS IoU는 예측끼리의 겹침, 평가 IoU는 예측과 정답의 겹침

2 하루 전체 지도 8교시가 네 묶음으로 이어진다

좌표·라벨·지표 무엇을 어떻게 적고 채점하나

- **1교시 객체탐지·BBox·좌표** "박스 숫자 4개는 어떤 형식인가?" — xyxy · cxcywh · 정규화 · BGR
- **2교시 IoU·NMS·P/R·mAP** "예측이 맞았다는 기준은 무엇인가?" — IoU · TP·FP·FN · mAP50-95
- **3교시 라벨·YOLO 포맷·감사** "라벨과 분할을 믿을 수 있나?" — class cx cy w h · dataset.yaml · 누수

→ **학습·검증** 만들고 고른다

- **4교시 YOLO 구조·학습 시작** "데이터와 모델이 제대로 연결됐나?" — 격자·후보 · .yaml vs .pt · train()
- **5교시 검증과 Checkpoint** "어떤 checkpoint를 무엇으로 고르나?" — val() · mAP50·50-95 · best·last

→ **추론·추적** 새 입력에서 읽는다

- **6교시 추론·Results·오류** "결과를 어떻게 읽고, 어떤 오류인가?" — predict() · conf · NMS IoU
- **7교시 다중 객체 추적·속도** "프레임 사이 같은 객체를 어떻게 잇나?" — track() · Track ID · persist

→ **통합**

- **8교시 탐지·추적 데모** "다시 실행해도 같은 결과인가?" — 재실행 · 기록

3 교시별 핵심 같은 개념이 난이도별로 어떻게 깊어지나

기본필수 동작을 확인한다 → 응용 바꿔 보고 잰다 → 심화 믿어도 되는지 확인한다

교시	질문	기본	응용	심화
1교시	**박스 숫자 4개는 어떤 형식인가?** xyxy · 중심 xywh · 정규화 · BGR	정답 박스 **그리기**	xyxy ↔ YOLO **왕복 오차**	경계 **clip**·작은 박스
2교시	**맞았다는 기준은?** IoU · 1:1 매칭 · TP·FP·FN · NMS · mAP	**IoU**와 TP·FP·FN 손계산	conf·**NMS IoU**를 바꿔 비교	클래스별 **AP**, mAP50 vs mAP50-95
3교시	**라벨·분할을 믿을 수 있나?** `class cx cy w h` · dataset.yaml · 누수 · Negative image	라벨 TXT **읽기**	면적·비율·**그룹 중복**·해시 감사	**그룹 분할** vs 이미지 분할
4교시	**데이터와 모델이 연결됐나?** 격자·후보 · Backbone·Neck·Head · `.yaml` vs `.pt`	1 epoch **smoke test**	seed·optimizer 고정, **API = CLI**	**imgsz** 하나만 바꿔 비교
5교시	**무엇으로 고르나?** `val()` · P·R·mAP50·mAP50-95 · best·last	기록표에서 **best epoch** 찾기	**클래스별** P·R·AP	**best vs last** 같은 val에서 비교
6교시	**결과를 어떻게 읽나?** Results · `boxes.xyxy/cls/conf` · conf · 오류 유형	predict → **CSV** (0개 포함)	**conf** 0.15·0.25·0.50 비교	오류를 **유형별로 분류**
7교시	**같은 객체를 어떻게 잇나?** `track()` · Track ID · persist · ID switch · 속도 범위	**ByteTrack**으로 추적 → CSV	**Tracker 2종** 비교·처리량	모듈형 프로젝트, **해시** 기록
8교시	**다시 해도 같은가?** 재실행 · 허용오차 · 기록 항목	내가 **재실행**해 대조	**짝이** README만 보고 재실행	**새 프로세스에서 복원** 대조

4 코드로 보는 하루 흐름 공식 문서의 기본 사용법 그대로, 오늘 배우는 과정을 가장 짧게 줄였습니다

① 좌표 1 — 픽셀 xyxy → box_convert로 중심·크기 → 이미지 크기로 나눔
→ ② 채점 기준 2 — box_iou로 예측 × 정답 겹침 표 → 0.5 이상이면 같은 객체
→ ③ 라벨·데이터 3 — 라벨 한 줄 = `class cx cy w h`, dataset.yaml로 경로·클래스 연결
→ ④ 학습 4 — 구조(.yaml)로 모델 만들기 → `train()` → best.pt·last.pt
→ ⑤ 검증 5 — `val()` → mAP50·mAP50-95
→ ⑥ 추론 6 — `predict(conf)` → 안에서 후보 420개 → conf → NMS → Results
→ ⑦ 추적 7 — `track(tracker)` → Track ID
→ ⑧ 재실행 8 — 같은 입력·설정으로 다시 실행해 이미지 수·프레임 수·지표 대조

```python
import cv2
import torch
from torchvision.ops import box_convert, box_iou
from ultralytics import YOLO
```

import 준비 OpenCV는 그리기, `torchvision.ops`는 박스 계산, `ultralytics`는 YOLO 학습·추론·추적을 맡습니다.

```python
boxes = torch.tensor([[12, 18, 52, 58]], dtype=torch.float32)  # x1, y1, x2, y2
cxcywh = box_convert(boxes, in_fmt="xyxy", out_fmt="cxcywh")
labels = cxcywh / torch.tensor([160, 128, 160, 128])           # x는 W, y는 H로
cv2.rectangle(image, (12, 18), (52, 58), (0, 255, 0), 2)      # 색은 BGR
```

1교시 ① 좌표 같은 박스를 픽셀 xyxy(모서리)에서 cxcywh(중심·크기)로 바꾸고, **x는 폭 W로, y는 높이 H로** 나눠 0~1로 만듭니다. OpenCV 색은 BGR 순서입니다.
문서: PyTorch torchvision.ops 문서 · OpenCV 5.0 Drawing Functions

```python
iou = box_iou(pred_boxes, gt_boxes)        # [예측 수, 정답 수] 겹침 표
matched = iou >= 0.5                       # 같은 클래스와 1:1로 연결되면 TP
```

2교시 ② 채점 기준 `box_iou`는 예측 × 정답 겹침 표를 만듭니다. 0.5 이상이고 클래스가 같으면, 점수 높은 예측부터 **정답 하나에 하나씩만** TP로 연결합니다.
문서: PyTorch torchvision.ops 문서

```yaml
# labels/train/img_000.txt  →  0 0.200000 0.296875 0.250000 0.312500
#                              class cx cy w h  (객체 없는 이미지는 빈 파일)

# dataset.yaml
path: data              # 데이터 폴더
train: images/train     # path 기준 상대 경로
val: images/val
names:
  0: square
  1: circle
  2: triangle
```

3교시 ③ 라벨·데이터 라벨 한 줄 = 객체 하나(`class cx cy w h`). dataset.yaml이 경로와 클래스 이름을 모델에 연결합니다. 객체가 없는 이미지는 빈 파일입니다.
문서: Ultralytics Datasets › Object Detection

```python
model = YOLO("yolo26n.yaml")
model.train(data="dataset.yaml", epochs=60, imgsz=160)
```

4교시 ④ 학습 `.yaml`은 구조만 만들어 처음부터 학습하고, `.pt`는 학습된 가중치까지 불러옵니다. 학습이 끝나면 best.pt·last.pt가 남습니다.
문서: Ultralytics Modes › Train

```python
metrics = model.val()
metrics.box.map50, metrics.box.map         # mAP50, mAP50-95
```

5교시 ⑤ 검증 학습 직후 `model`은 best.pt입니다. 인자 없이 `val()`을 부르면 학습한 데이터 설정 그대로 val을 채점합니다.
문서: Ultralytics Modes › Val

```python
results = model.predict("images/val", conf=0.25)
for result in results:
    result.boxes.xyxy, result.boxes.cls, result.boxes.conf  # 0개일 수도 있음
```

6교시 ⑥ 추론 안에서 후보 420개 → conf로 거름 → NMS로 중복 제거 → Results. 탐지가 0개인 이미지도 결과에 남습니다.
문서: Ultralytics Modes › Predict

```python
results = model.track("demo.avi", tracker="bytetrack.yaml", stream=True)
for result in results:
    if result.boxes and result.boxes.is_track:
        track_ids = result.boxes.id.int().cpu().tolist()
```

7교시 ⑦ 추적 Tracker가 프레임 사이 같은 객체에 Track ID를 붙입니다. ID가 아직 없는 프레임이 있으니 `is_track`을 먼저 확인합니다.
문서: Ultralytics Modes › Track

`image`·`pred_boxes`·`gt_boxes`는 이미 준비된 이미지 배열과 박스 텐서입니다. 8교시는 새 코드 없이 같은 명령을 다시 실행해 이미지 수·프레임 수·지표를 대조합니다. 모든 줄은 Ultralytics 8.4.203에서 실제로 실행해 확인했습니다.

코드 속 파라미터 · 어디를 고치면 무엇이 달라지나

왼쪽은 위 핵심 코드 그대로, 가운데는 표시한 부분만 고친 코드입니다. 고친 코드로 다시 실행한 결과를 오른쪽에 적었습니다. **꼭 기억** 표시가 붙은 7개는 오늘 반드시 가져갈 결과입니다.

위 코드 (핵심 코드 그대로) → 고친 코드 (이 부분만 바꿔 다시 실행) → 결과

숫자·옵션만 바꾸기 줄은 그대로 두고 값만 고쳤습니다

6교시 conf

```python
model.predict("images/val", conf=0.25)
```
→
```python
model.predict("images/val", conf=0.5)
```

Precision **0.70 → 0.86** 남는 박스 43 → 35, Recall 1 그대로. 0.9까지 올리면 Recall 0.4(30개 중 18개 놓침)
응용 val 12장 · 평가 IoU 0.5 · 깔때기의 첫 거름망

꼭 기억 올리면 거짓 탐지는 줄고 놓침은 늘어난다. Precision과 Recall을 함께 본다

6교시 NMS IoU

```python
model.predict("images/val", conf=0.25)
```
→
```python
model.predict("images/val", conf=0.25,
              iou=0.5)  # 기본 0.7
```

FP **13 → 0** 남는 박스 43 → 30. conf 0.25에서 남은 FP 13개가 전부 같은 객체의 중복이었음. 0.95로 올리면 105개
응용 val 12장 · 깔때기의 두 번째 거름망

꼭 기억 NMS IoU는 예측끼리의 겹침이다. 정답과 비교하는 평가 IoU와 이름만 같고 하는 일이 다르다

2·5교시 평가 IoU

```python
matched = iou >= 0.5
```
→
```python
matched = iou >= 0.95
```

TP **30 → 1** 같은 예측 43개인데 거의 다 FP가 됨. 같은 모델의 mAP50 0.977, mAP50-95 0.710
응용 val 12장 · conf 0.25

꼭 기억 기준이 엄격할수록 위치가 정확해야 TP다. mAP50-95가 낮으면 "찾긴 하지만 박스가 덜 맞다"

4교시 epochs

```python
model.train(data="dataset.yaml",
epochs=60, imgsz=160)
```
→
```python
model.train(data="dataset.yaml", epochs=1,
imgsz=160)
```

mAP50 **0.977 → 0** 60 epoch 학습에서도 17 epoch까지는 0이었음
응용 데이터 · 처음부터 학습

꼭 기억 1 epoch이 돌아간 것은 "연결 성공"이지 "탐지 성공"이 아니다

7교시 persist

```python
model.track("demo.avi",
            tracker="bytetrack.yaml",
            stream=True)
```
→
```python
model.track("demo2.avi",  # 다음 영상
            tracker="bytetrack.yaml",
            stream=True, persist=True)
```

가장 큰 ID **26 → 45** 새 영상인데 앞 영상의 ID(1·3·6·24·26)를 이어 씀. 새 YOLO 객체로 시작하면 첫 실행과 같은 ID
같은 모델 객체로 demo 영상을 다음 영상처럼 한 번 더

꼭 기억 Track ID는 모델에 저장된 값이 아니라 실행 중 상태다. persist는 같은 영상 안에서만 쓴다

순서만 바꾸기 같은 값을 다른 자리에 넣었습니다 (흔한 실수)

1교시 W·H 순서

```python
labels = cxcywh / torch.tensor([160,
128, 160, 128])
```
→
```python
labels = cxcywh / torch.tensor([128, 160,
128, 160])
```

IoU **1.0 → 0.49 · 0.15 · 0.06** 라벨을 다시 픽셀로 되돌려 원래 박스와 비교. 오른쪽 박스는 이미지 밖으로 나감
손 예제(정답 박스 3개)

꼭 기억 x는 W로, y는 H로 나눈다. 저장한 라벨은 다시 그려서 확인한다

학습 코드는 그대로, 나누고 고르는 방법만 바꾸기 데이터를 나누는 단위나 남길 checkpoint만 바꿨습니다

3교시 분할 단위

```python
split = "train" if group_id in
train_groups else "val"
```
→
```python
# 이미지를 번갈아 배정
split = "train" if i % 2 == 0 else "val"
```

split을 넘나드는 그룹 **0 → 24** 24개 그룹 전부가 train과 val에 걸침
응용 데이터 48장(24그룹)

꼭 기억 같은 장면·연속 프레임이 양쪽에 섞이면 점수가 부풀려진다. 그룹 단위로 나눈다

5교시 checkpoint

```python
metrics = model.val()  # 학습 뒤 model =
best.pt
```
→
```python
model = YOLO(
    "runs/detect/train/weights/last.pt")
metrics = model.val()
```

mAP50-95 **0.710 → 0.688** mAP50은 오히려 0.977 → 0.986. best(50 epoch)는 mAP50-95가 가장 높은 epoch
응용 val · last = 60 epoch

응용·심화는 무엇을 더 요구하나 세 난이도는 같은 코드 흐름을 쓰지만, 요구하는 수준이 다릅니다.

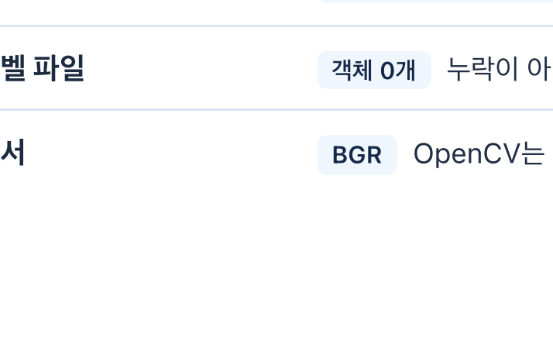
- **기본필수 어떻게 동작하나?** 흐름을 끝까지 돌리고, 각 출력의 의미를 안다 — 결과를 보는 눈 · 박스가 나오나
- → **응용 무엇을 바꾸면 결과가 달라지나?** 값 하나를 바꿔 결과 차이를 **재고 설명**한다 — 결과를 보는 눈 · 숫자가 얼마나 달라지나
- → **심화 그 결론을 믿어도 되나?** 그 차이가 **누수·설정·우연 탓이 아님**을 보인다 — 결과를 보는 눈 · 다른 조건·새 프로세스에서도 같은 결론인가

	응용이 더 요구하는 것	심화가 더 요구하는 것
1교시 좌표	**왕복 변환** 오차 확인 저장한 숫자를 다시 그려야 틀린 정규화가 드러남	경계 **clip**과 작은 박스 이미지 밖 좌표는 면적이 달라져 IoU가 바뀜
2교시 지표	conf와 **NMS IoU**를 따로 바꿔 비교 두 값은 깔때기의 서로 다른 단계를 바꿈	**클래스별 AP**와 mAP50-95 평균 하나 뒤에 0점인 클래스가 숨을 수 있음
3교시 데이터	면적·비율·**그룹 중복**·해시 감사 빈 라벨은 누락이 아니라 정상 기록	**그룹 분할** 이미지 단위로 나누면 같은 장면이 train·val에 섞임
4교시 학습	seed·optimizer를 고정하고 **API와 CLI를 같은 설정**으로 기록	**imgsz 하나만** 바꿔 파라미터 수·시간·지표 비교
5교시 검증	**클래스별** P·R·AP로 먼저 볼 오류 이미지 정하기	**best와 last**를 같은 val에서 비교 마지막 epoch이 최선이 아닐 수 있음
6교시 추론	**conf 후보 3개**로 TP·FP·FN 비교 올리면 FP↓, FN↑	오류를 **유형별 분류**(중복·배경 FP·놓침 등)하고 우선 개선안 작성
7교시 추적	**Tracker 2종**을 독립 인스턴스로 비교, 처리량과 측정 범위 기록	모듈형 프로젝트로 나누고 **모델 해시·처리 수**를 기록
8교시 재현	**짝이** README만 보고 재실행	**새 프로세스에서 복원**해 행 수·좌표·확률을 허용오차로 대조

5 전체 파라미터 지도 "무엇을 정하는 값인가"로 묶었습니다. 그림은 실제 실행 결과입니다

A 좌표와 라벨 숫자 4개가 무엇을 뜻하나

W·H를 뒤바꿔 정규화하면 박스가 어긋난다 (손 예제, 160×128)



● 실선 = 원래 정답 박스
● 점선 = `[128, 160, …]`로 나눈 라벨을 다시 픽셀로 되돌린 박스

폭 160과 높이 128이 바뀌어 위치와 크기가 함께 틀어집니다. 오른쪽 박스는 이미지 밖으로 나갑니다.

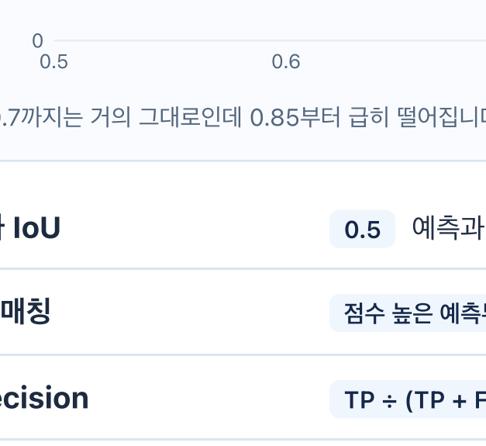
맞는 라벨 한 줄

```
0 0.200000 0.296875 0.250000
0.312500
```
class cx cy w h

xyxy	`[x1, y1, x2, y2]` 픽셀	왼쪽 위·오른쪽 아래. 그리기·IoU 계산용
cxcywh	`[cx, cy, w, h]` 픽셀	중심과 크기. Ultralytics `boxes.xywh` 형식
정규화 좌표	0~1 (x÷W, y÷H)	해상도와 무관한 라벨 저장 형식
라벨 한 줄	`class_id cx cy w h`	객체 하나 = 한 줄, 숫자 5개
빈 라벨 파일	객체 0개	누락이 아니라 "없음"을 적은 정상 기록
색 순서	BGR	OpenCV는 파랑·초록·빨강 순서

B 채점 기준 맞았다는 기준은 무엇인가

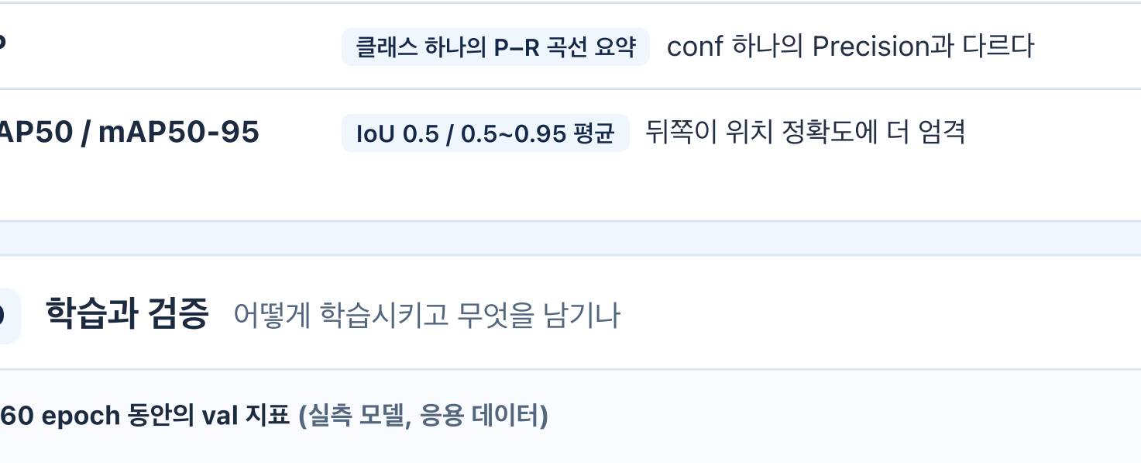
1:1 매칭 · 예측 4개와 정답 3개 (손 예제, 평가 IoU 0.5)



	conf	최대 IoU	판정
①	0.95	IoU 0.90	TP
②	0.85	IoU 0.82	FP 같은 정답을 이미 ①이 차지 → 중복
③	0.80	IoU 1.00	TP
④	0.40	IoU 0.00	FP 겹치는 정답 없음 → 배경

TP 2 · FP 2 · FN 1 → Precision 0.50, Recall 0.67

기준을 엄격하게 하면 mAP가 떨어진다 (실측 모델, 응용 val)

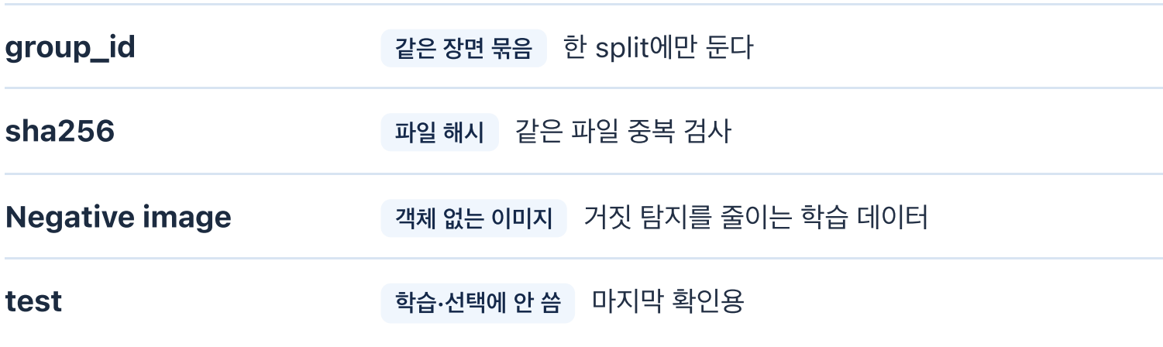


0.7까지는 거의 그대로인데 0.85부터 급히 떨어집니다. 객체는 찾지만 박스 위치가 아주 정확하지는 않다는 뜻입니다.

평가 IoU	0.5	예측과 정답이 이만큼 겹쳐야 같은 객체
1:1 매칭	점수 높은 예측부터	한 정답에는 하나만 TP, 나머지는 FP
Precision	TP ÷ (TP + FP)	탐지한 것 중 맞은 비율
Recall	TP ÷ (TP + FN)	실제 객체 중 찾은 비율
AP	클래스 하나의 P–R 곡선 요약	conf 하나의 Precision과 다르다
mAP50 / mAP50-95	IoU 0.5 / 0.5~0.95 평균	뒤쪽이 위치 정확도에 더 엄격

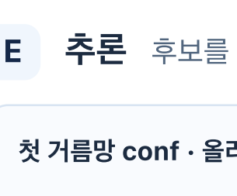
C 데이터 계약 데이터를 믿을 수 있나

누수를 막는 그룹 분할 (응용 데이터 train+val 48장, 24그룹)


이미지 번갈아 ✕ 24개 그룹이 train·val에 걸침
그룹 단위 ✓ 걸치는 그룹 0개

점선 상자 = 같은 장면에서 나온 이미지 2장(그룹). 🟦 train 🟧 val

빈 라벨 = 정상 기록

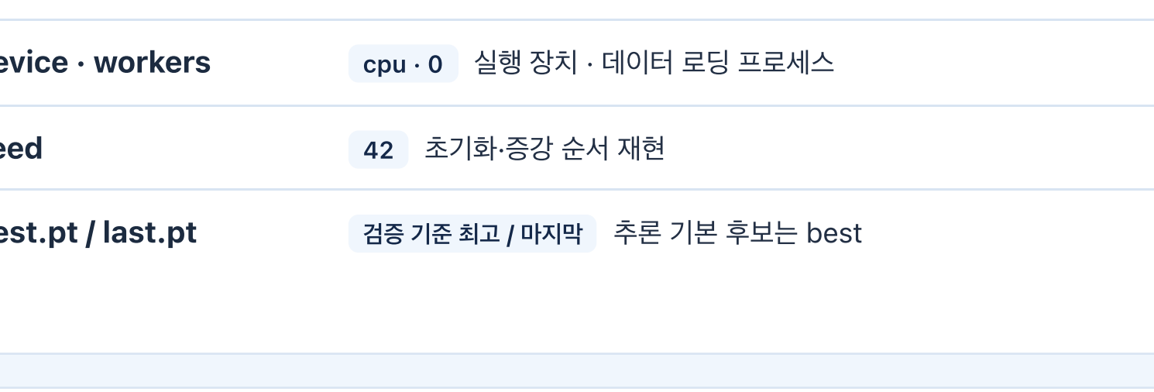


객체 없는 이미지도 학습 데이터입니다.
라벨 파일은 있고 내용은 비어 있음(0 byte) = "객체 0개"라는 기록. 파일이 아예 없는 누락과 다릅니다.
응용 데이터: train 6장 · val 2장 · test 2장

dataset.yaml	path · train · val · names	데이터와 모델을 잇는 계약서
names	0 square, 1 circle, 2 triangle	class_id ↔ 이름
group_id	같은 장면 묶음	한 split에만 둔다
sha256	파일 해시	같은 파일 중복 검사
Negative image	객체 없는 이미지	거짓 탐지를 줄이는 학습 데이터
test	학습·선택에 안 씀	마지막 확인용

D 학습과 검증 어떻게 학습시키고 무엇을 남기나

60 epoch 동안의 val 지표 (실측 모델, 응용 데이터)

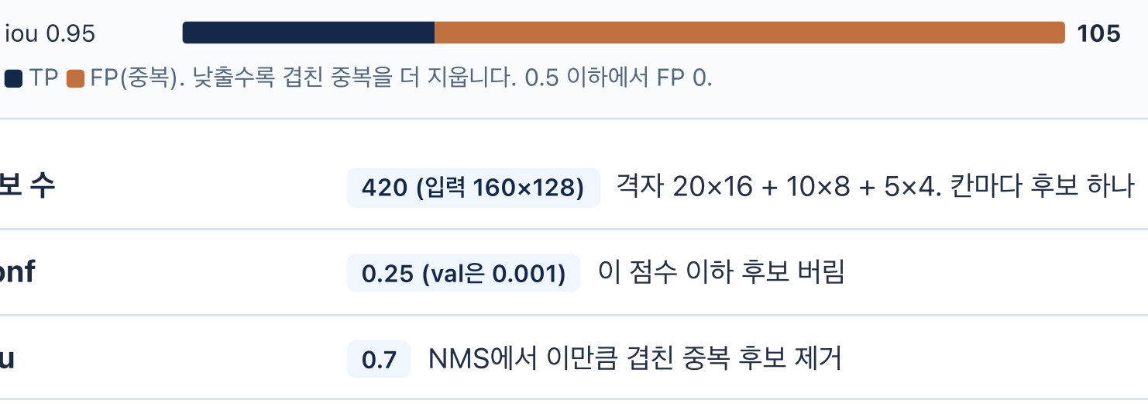


처음 17 epoch은 박스를 하나도 맞히지 못했습니다. epochs=1로 따로 학습해도 mAP는 0입니다. best.pt는 mAP50-95가 가장 높았던 50 epoch이고, 마지막 epoch과 다릅니다.

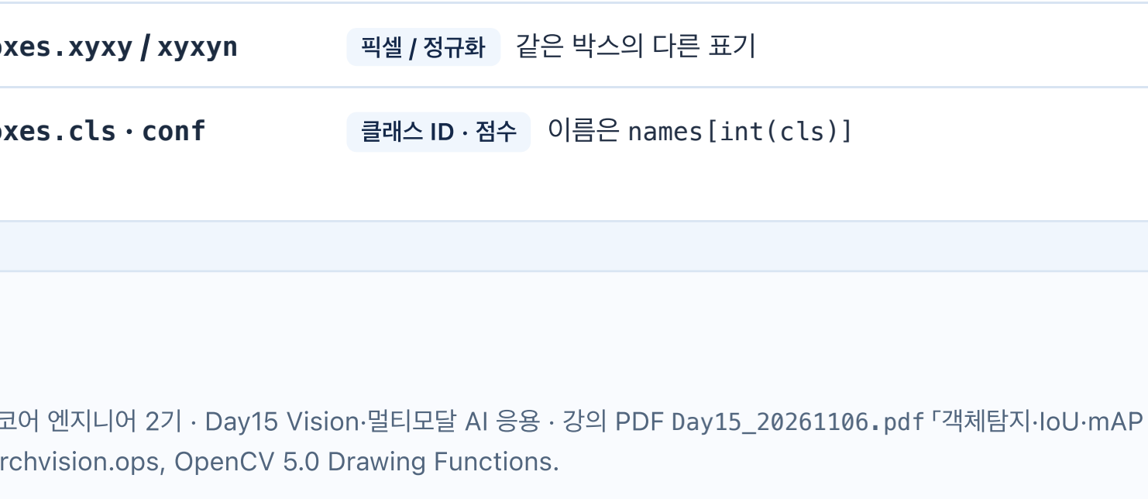
model	`yolo26n.yaml` / `best.pt`	구조만 / 학습된 가중치까지
epochs	1(점검) · 60(학습)	전체 train 반복 횟수
imgsz	160	입력 크기. 격자 크기와 작은 객체·속도·메모리를 함께 바꾼다
batch	4	한 번에 넣는 이미지 수
device · workers	cpu · 0	실행 장치 · 데이터 로딩 프로세스
seed	42	초기화·증강 순서 재현
best.pt / last.pt	검증 기준 최고 / 마지막	추론 기본 후보는 best

E 추론 후보를 어떻게 거르고 결과를 어떻게 읽나

첫 거름망 conf · 올리면 Precision↑ Recall↓ (응용 val 12장)



두 번째 거름망 NMS IoU · 남는 박스 수 (conf 0.25)

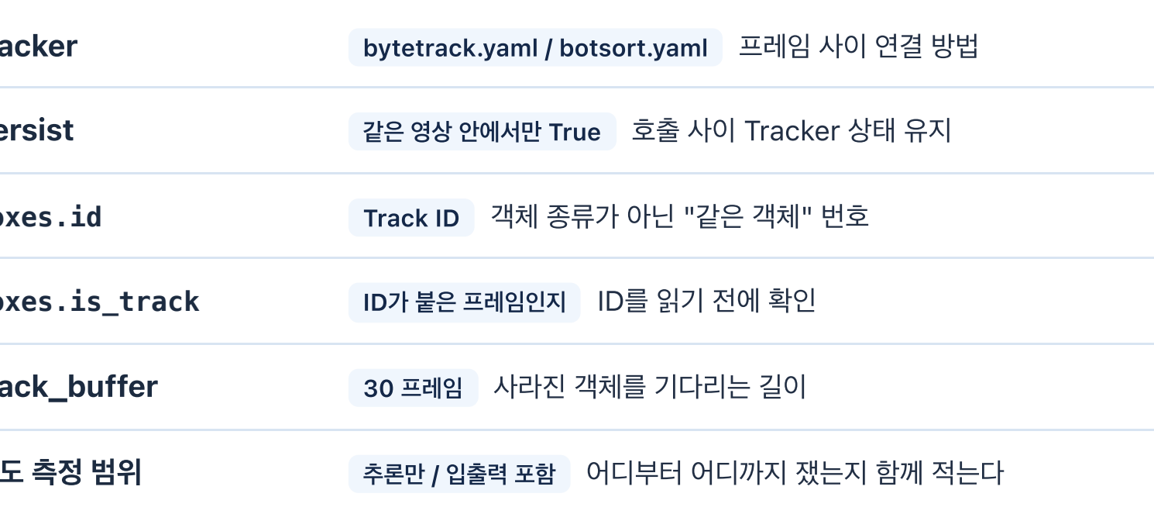


■ TP ■ FP(중복). 낮출수록 겹친 중복을 더 지웁니다. 0.5 이하에서 FP 0.

후보 수	420 (입력 160×128)	격자 20×16 + 10×8 + 5×4. 칸마다 후보 하나
conf	0.25 (val은 0.001)	이 점수 이하 후보 버림
iou	0.7	NMS에서 이만큼 겹친 중복 후보 제거
stream	True(영상)	결과를 한 장씩 생성, 메모리 절약
`boxes.xyxy / xyxyn`	픽셀 / 정규화	같은 박스의 다른 표기
`boxes.cls · conf`	클래스 ID · 점수	이름은 `names[int(cls)]`

F 추적 ID를 어떻게 잇나

실제 객체 3개 → Track ID 11개 (demo 영상 30프레임, ByteTrack)



한 객체가 여러 ID로 쪼개지고(fragmentation), 겹친 중복 박스에도 ID가 붙습니다. ID 개수는 객체 수가 아닙니다. 추적 품질은 탐지 품질에 묶입니다.

Track ID는 실행 중 상태

새 YOLO 객체로 다음 영상 1 2 3 4 5 6 9 16 21 24 26
같은 객체 + `persist=True` 1 3 6 24 26 28 29 34 35 40 43 45

같은 영상을 다음 영상처럼 넣은 실험입니다. persist를 잘못 쓰면 앞 영상의 상태를 이어받아 ID가 이어 붙고 커집니다.

tracker	bytetrack.yaml / botsort.yaml	프레임 사이 연결 방법
persist	같은 영상 안에서만 True	호출 사이 Tracker 상태 유지
`boxes.id`	Track ID	객체 종류가 아닌 "같은 객체" 번호
`boxes.is_track`	ID가 붙은 프레임인지	ID를 읽기 전에 확인
track_buffer	30 프레임	사라진 객체를 기다리는 길이
속도 측정 범위	추론만 / 입출력 포함	어디부터 어디까지 쟀는지 함께 적는다

출처

NVIDIA AI 코어 엔지니어 2기 · Day15 Vision·멀티모달 AI 응용 · 강의 PDF `Day15_20261106.pdf` 「객체탐지·IoU·mAP → YOLO 학습·추론 → 추적·결과 분석」(106쪽). 코드 문법: Ultralytics 문서(Train·Val·Predict·Track, Object Detection Datasets), PyTorch torchvision.ops, OpenCV 5.0 Drawing Functions.

그림과 숫자는 응용 실습 데이터 train 36장으로 `yolo26n.yaml`을 60 epoch 학습한 모델로 직접 실행한 결과입니다(imgsz 160, seed 42, CPU · Ultralytics 8.4.203, torch 2.14.0, OpenCV 5.0.0). 실습의 `data/demo.pt`와 숫자가 다를 수 있습니다.

합성 도형 데이터라 쉬운 문제입니다. 실제 사람·차량 탐지 성능을 뜻하지 않습니다.